Day 3 – 'Github Copilot'

Recap – Day 2

① 'MCP' → Extending the capabilities of AI Assistant – providing Access to our KB, internal system, etc.

② use cases – Code Review, multi-file Reasoning & Context linking
– Maintain the Consistency Across the team (Large Code Base)
.github / copilot-instructions.md

③ Agent Mode (Coding Agent)

How?
@ "PROMPT" → Context Collection ✓ → Plan ✓ → Human Review → Execution → Diff → Review

How to MINIMIZE Token usage?
① Provide the RIGHT Context Window.
② Limit file References.
③ Incremental Execution ✓
④ Plan first → step-by-step
⑤ Control the output size

Agent Mode / "Coding Agent"

Cloud Agent ⇒ official name

Copilot chat / Inline Copilot (Chat) ⇒ You're coding

✓ Copilot chat (Agent Mode) – IDE
↳ Helps you make multiple files changes

✓ Copilot cloud agent → More independently in the cloud.
→ Assign it a task

Typical workflow (Github Repo)
① Dev creates / select a Github issue
② Issue assigned to Copilot Cloud agent
③ Copilot studies the Repo & create a plan
④ Copilot makes code changes on a separate branch.
⑤ Copilot opens a PR
⑥ Dev reviews the diff, test the changes, request updates if needed
⑦ Human reviewer approves & merges only after validation

Good use cases: (Cloud Agent)
① Bug fix = fix the login validation bug described in the issue & add regression tests.
② feature implementation = Add pagination to the Products
③ Refactoring
④ Test generation
⑤ Documentation
⑥ Small modernization

Not Ideal / Avoid – Cloud Agent
① Sensitive production Hotfixes with Review ↵
② Poorly described Tasks.
③ Tasks needing private business context not in repo/issues
④ Major Architecture Design
⑤ Security - sensitive changes.

Lab: Assign a Github issue to Copilot Cloud Agent
① open a sample Github Repo (MCP Demo)
② Create an issue:
"Add input validation to the Products."
③ Assign the issue to Copilot
④ Review the plan generated by Copilot] * ✓
⑤ let Copilot create a branch & PR.
⑥ Review the changed files
⑦ Ask Copilot to make improvements in PR Comments] * ✓
⑧ Show the final merge decision remains.] * ✓

Strategic Adoption & ROI

① How does Copilot fit into Devops workflow?
② How do we know it is actually HELPING?
③ How do we avoid BAD Habits?
④ How do you keep updated / current as Copilot changes?

Plan → Code → Test → Review → Secure → Document → Deploy → learn

① Copilot in the Planning Stage.
↳ Reason, plan, identify risks
↳ THINKING Assistant Before Coding
② Copilot in the Coding Stage
③ Testing.
④ Pull Requests & Code Reviews
⑤ Documentation
⑥ Copilot in Devops & Pipeline Support

Measuring Productivity Gains

ROI ≠ Copilot write a lot of code
↳ Deliver useful, secure, maintainable S/W faster.

Productivity Metrics

Metric →	Before Copilot	After Copilot
Avg time to first PR		
PR Cycle Time		
Code Review TRT		
Test coverage		
Deployment freq		
Lead time for changes		

ROI = Value Gained – "Cost of Adoption"

Value Gained = Time Saved + Quality + faster onboarding + reduced context switching + improved test coverage + faster documentation

Cost of Adoption = license cost + TRG time + governance setup + change mgmt + Review overhead.

50 devs = Copilot = ROI?

Dev = 50
Avg coding Hrs / week = 20
~~Conserv~~ Conservative time saving = 5%
Hours saved / dev / week = 1 hr
Total # Hrs / week = 50
Total # Hrs / month = 200

Keeping Skills Current as Copilot Evolves.

↳ Reference links → Official Docs – Github Copilot

Team A = Vishal, Saurabh, Rutik, Kushal
Team B = Kiran, Sutish, Imran
Team C = Ramya, Shreya, Ajinkya